

Week 2 Report

Z24 PDT Model-Ready Dataset and Classification Pipelines

TRAN VAN PHUONG

20 September 2026

Abstract

This report documents the second-week work on the Z24 Progressive Damage Test (PDT) training data. The cleaned CSV files are converted into fixed-length time-series windows, normalized using training statistics, and passed to two classification pipelines: the teacher’s 1DCNN-LSTM-ResNet implementation and a tsai ResNet baseline. The report describes the input contract, function flow, mock examples, split strategy, model interfaces, experiment results, and the current overfitting diagnosis.

1 Research objective

The task is supervised time-series classification:

Input X = synchronized sensor values over time Output y = one of the 17 Z24 bridge conditions
--

Condition directories 01–17 are mapped to zero-based labels 0–16. The zero-based representation is used by the classification loss, while the original condition numbers remain available in the manifest and report files.

2 Clean-data source and model contract

The training notebooks read the newest complete run under `processed/avt/` or `processed/fvt/`. The current AVT run contains 153 source recordings and 1,683 CSV segments. Each source recording is represented by a unique `source` value in `manifest.json`.

The documented sampling frequency is 100 Hz. The model window is 8,000 samples, corresponding to 80 seconds. The five channels present in every recording are:

R1V, R2L, R2T, R2V, R3V

The cleaner preserves 33 channels in the CSV files, but the training loader uses the intersection of channel names across all recordings. This common-channel rule currently produces five channels and gives a consistent input layout for every window.

3 Function flow in `pdt_training_data.py`

3.1 `latest_complete_run(project_root, measurement)`

Target. Select the newest timestamped clean-data directory whose `report.json` and `manifest.json` exist and whose report status is `complete`.

Mock input.

```
run_dir = latest_complete_run(
    r"C:\\Users\\phuotran\\Data\\research\\shm",
    measurement="avt",
)
```

Mock output.

```
C:\Users\phuotran\Data\research\shm\processed\avt\
avt_13-09-2026_21-52-09
```

The function only selects a directory. It does not read sensor values or modify any data.

3.2 read_manifest(run_dir)

Target. Read manifest.json and convert its JSON content into a Python list of dictionaries.

Mock input.

```
records = read_manifest(run_dir)
```

Mock output.

```
[
  {
    "file": "segments/condition01_setup01_avt_segment00.csv",
    "condition_id": 1,
    "condition_name": "undamaged_reference_1",
    "label": 0,
    "setup_id": 1,
    "measurement": "avt",
    "segment": 0,
    "start_sample": 0,
    "samples": 6000,
    "channels": 33,
    "channel_names": ["99V", "100V", "R1V", "R2L", "R2T", "R2V", "R3V"]
  }
]
```

The manifest is metadata. It tells the loader which CSV belongs to which condition, setup, recording, and segment.

3.3 common_channels(records)

Target. Find the sensor names present in every recording and preserve their order from the first recording.

Mock input.

```
Recording 1: [A, B, C, D]
Recording 2: [B, C, D, E]
Recording 3: [C, D, F]
```

Flow.

```
common = names from Recording 1
common &= names from Recording 2
common &= names from Recording 3
```

Mock output.

```
("C", "D")
```

For the Z24 AVT run, the result is R1V, R2L, R2T, R2V, R3V.

3.4 `split_by_recording(records, seed)`

Target. Create train, validation, and test lists while keeping every source recording in exactly one split.

The function groups source recordings by label, shuffles them with a reproducible seed, and assigns the requested fractions. Because every condition currently has nine recordings, the resulting allocation is:

```
6 recordings per condition -> train
1 recording per condition -> validation
2 recordings per condition -> test
```

This is approximately 66.7%, 11.1%, and 22.2% within each condition. The nominal function arguments are 15% validation and 20% test, but the minimum-one-recording rule determines the exact allocation for nine recordings.

Mock output.

```
{
  "train": [manifest rows from 102 recordings],
  "validation": [manifest rows from 17 recordings],
  "test": [manifest rows from 34 recordings]
}
```

The disjointness assertions verify that no source recording appears in two splits.

3.5 `_portable_relative_path(value)`

Target. Convert either Windows or POSIX separators into a portable relative `Path`.

Mock input and output.

```
_portable_relative_path("segments\\condition01_setup01_avt_segment00.csv")
# segments/condition01_setup01_avt_segment00.csv
```

This allows the same manifest to be read on Windows, Linux, Kaggle, or Colab.

3.6 `load_windows(run_dir, records, channel_names, window_samples)`

Target. Join all CSV storage segments belonging to each source recording and create fixed-length model windows.

Mock input.

```
sample_index,R1V,R2V,sensor_X
0,10,20,100
1,11,21,101
2,12,22,102
3,13,23,103
4,14,24,104
5,15,25,105
```

If `channel_names = [R1V, R2V]` and `window_samples = 3`, the output is:

```
X = [
    [[10, 20], [11, 21], [12, 22]],
    [[13, 23], [14, 24], [15, 25]]
]
y = [condition_label, condition_label]
```

The output shape is (number_of_windows, window_samples, number_of_channels) before backend-specific conversion. A remainder shorter than the requested window length is discarded; data from two different recordings is never joined.

For the current AVT dataset:

```
X_train = (816, 8000, 5)
X_validation = (136, 8000, 5)
X_test = (272, 8000, 5)
```

The 816 value is the number of train windows, not the number of raw sensor samples. The complete dataset contains 1,224 usable 8,000-sample windows across all three splits.

3.7 normalize_from_train(x_train, *other_arrays)

Target. Put all channels on comparable numerical scales without using validation or test statistics.

For every channel, the function calculates:

```
normalized_value = (value - training_mean) / training_std
```

The same training mean and standard deviation are applied to validation and test arrays. This prevents preprocessing information from the test set entering the model-training process.

4 End-to-end data flow

```
processed/avt/
|
v
latest_complete_run()
|
v
read_manifest()
|
v
common_channels() -> five shared sensors
|
v
split_by_recording()
|
+--> train: 102 recordings, 816 windows
+--> validation: 17 recordings, 136 windows
+--> test: 34 recordings, 272 windows
|
v
load_windows()
|
v
normalize_from_train()
|
+--> TensorFlow input for 1DCNN-LSTM-ResNet
|
```

```
+--> tsai input: (samples, channels, timesteps)
```

5 The two model pipelines

5.1 Teacher-compatible 1DCNN-LSTM-ResNet

The notebook imports `build_model()` from `src/models/dcn_lstm_resnet.py`. The model contains an initial Conv1D layer, two residual blocks, a 128-unit LSTM, two shortcut paths, global average and maximum pooling, a dense layer, and a 17-class softmax output.

The training notebook creates TensorFlow datasets with batch size 64, trains for at most 100 epochs, monitors validation accuracy, and saves the best checkpoint. The current project implementation additionally contains Dropout, L2 regularization, and AdamW. Therefore its backbone follows the teacher source, but the complete training model is not identical to the teacher's original code.

Input-layout note. The current notebook transposes the arrays to `(samples, 5, 8000)` to match the teacher source. Standard Keras Conv1D convention is `(samples, timesteps, channels)`, or `(samples, 8000, 5)` for this data. This layout must be resolved before reporting final DCNN results.

5.2 tsai ResNet baseline

tsai uses the contract:

```
samples x variables x sequence_length
```

Therefore the arrays are converted to:

```
X_train = (816, 5, 8000)
X_validation = (136, 5, 8000)
X_test = (272, 5, 8000)
```

Train and validation arrays are combined with explicit split indices for `TSClassifier`. The test set remains outside the learner until final evaluation. The baseline uses tsai's ResNet architecture, a batch size of 32, and the fastai one-cycle training schedule.

6 Observed results and overfitting diagnosis

The recorded runs show a large gap between training and evaluation performance:

Pipeline	Training	Validation	Test
DCNN-LSTM-ResNet	96.45%	16.18%	15.81%
tsai ResNet	95.59%	9.56%	23.16%

The main hypotheses are:

1. There are only six independent training recordings per condition. The eight windows generated from one recording are correlated and should not be counted as eight independent experiments.
2. The DCNN implementation currently has a channel/time input-layout problem. Passing `(5, 8000)` to Keras Conv1D makes the network interpret five as the time length and 8,000 as the channel count, which greatly increases the first-layer parameter count.
3. Only five of the 33 available sensor columns are shared by every recording. The excluded channels may contain useful damage information, while the remaining channels may not fully separate all 17 conditions.

4. An 80-second window can contain long periods of low-information vibration. Assigning the recording label to every window may create weakly informative or noisy training targets.
5. Validation contains only one recording per condition, so its accuracy has high variance. The higher tsai test accuracy is not evidence that validation is better; it reflects the small grouped evaluation sample and a different pair of recordings.

The split-by-recording design prevents direct segment leakage, but it does not create more independent recordings. The next experiments should correct the Keras input layout, use early stopping on validation loss, report recording-level majority-vote accuracy, and compare grouped cross-validation results. Additional experiments can evaluate per-recording detrending, controlled time-series augmentation, and a principled method for using the non-common sensors.

7 Conclusion

Week 2 established the model-ready time-series interface for the cleaned Z24 dataset and implemented two comparable classification backends. The main deliverable is a reproducible path from manifest metadata to fixed windows, training-only normalization, model training, and final evaluation. The current metrics demonstrate that the models can fit the training windows but do not yet generalize reliably to unseen recordings. Correcting the DCNN input layout and evaluating the limited recording-level sample size are prerequisites for a defensible model comparison.